

PAB-Spline VLA: Parallel Agent B-Spline Tokenization for Generalist Humanoid Vision-Language-Action Models

Debbie Yuen · Huu Nguyen · Van Khue Nguyen · Andreas Koepf

Technical Specification v0.1 — May 2026

Abstract

We present PAB-Spline VLA, a framework for training generalist humanoid Vision-Language-Action (VLA) models using a unified parametric token representation. The core contribution is a universal tokenizer that converts heterogeneous input/output modalities — proprioceptive joint states, Cartesian FK positions, video (via Cosmo/VAE latents), audio (frequency domain), and language — into a single interleaved token stream consumable by a standard large language model (LLM). Action signals (joint angles q , velocities $\dot{q}$, and pelvis-relative positions x,y,z) are compressed using adaptive interpolating B-splines parameterized by a phase variable $\phi \in [0,1]$, enabling temporal scaling and cyclic trajectory representation. Static joints are detected and encoded as single-token constants, achieving 7–15× compression over raw timestep vectors. Perception signals (video frames, goal images, subgoal images, audio) are tokenized as Cosmo tokens and interleaved with bundled pose tokens on a common time grid. A multi-agent OS architecture (Safety, Motion, Vision, Exploration agents) enables real-time trajectory preemption without VLM re-inference. The framework is source-agnostic: tokens may originate from Isaac Sim simulation, RL policy outputs, motion capture retargeting, or real hardware sensors.

1. Introduction and Motivation

Current VLA architectures (RT-2, OpenVLA, Octo, BEAST) share a fundamental limitation: they tokenize actions as discrete coordinate bins or raw delta vectors, producing jerky, discontinuous motion that requires high-frequency inference. This creates three unsolved problems:

- **Smoothness:** Discrete tokens cannot guarantee C^2 continuity across trajectory chunks.
- **Compression:** Full-state prediction at every timestep wastes capacity on stationary joints.
- **Safety latency:** End-to-end VLM re-inference (200–500ms) is unacceptable for real-time preemption.

PAB-Spline VLA addresses all three through parametric B-spline tokenization combined with a parallel multi-agent architecture. The key insight is that **gait, audio, and video are all periodic or smooth signals** — they compress efficiently with splines, and an LLM can learn to predict spline tokens the same way it predicts text tokens. The source of the signal does not matter: the same tokenizer handles Isaac Sim rollouts, RL policy outputs, real hardware IMU data, and MoCap retargeting.

2. System Architecture

The system consists of four layers:

Layer	Components	Description
Perception	Camera(s), Audio, Proprio	Raw sensor streams at native rates
Tokenization	Cosmo Tokenizer PAB-Spline Tokenizer	Convert all modalities to unified tokens
LLM Core	Generalist VLA model	Predicts next token(s) given context window
Execution	Multi-Agent OS (Safety, Motion, Vision, Exploration)	Decodes tokens → robot commands with preemption

2.1 Token Stream Structure

The LLM receives a single interleaved token stream per time segment:

```
[STATIC: left_shoulder=0.0, t=0→4.67s] (upfront, joints not moving this segment)  
[LANG: "walk forward and turn right"]  
[COSMO: video_t=0.00] [POSE:  $\phi=0.00$ , all_joints]  
[COSMO: video_t=0.03] [POSE:  $\phi=0.18$ , all_joints]  
[COSMO: video_t=0.07] [POSE:  $\phi=0.42$ , all_joints]  
[COSMO: subgoal_img] [POSE:  $\phi=0.61$ , all_joints]  
[COSMO: goal_img] [POSE:  $\phi=1.00$ , all_joints]
```

Static joints are omitted from POSE tokens entirely — one upfront static token covers the full segment duration. This is the primary compression mechanism for tasks where only a subset of joints are active (e.g. walking: legs active, arms static → ~8–12× reduction in pose token size).

3. PAB-Spline Action Tokenizer

3.1 Signal Types

Per joint, the following signals are tokenized (configurable via TokenizerConfig):

Signal	Units	Description	Typical thresh
q	radians	Joint angle	0.01 rad
qd	rad/s	Joint velocity	0.05 rad/s
x	metres	Pelvis-relative FK position X	0.005 m
y	metres	Pelvis-relative FK position Y	0.005 m
z	metres	Pelvis-relative FK position Z	0.005 m

3.2 Token Types

3.2.1 Static Token

Emitted when a signal's range over the segment is below the static threshold. Encodes the mean value and the full time range in a single token:

```
{ "type": "static", "joint": "left_shoulder_pitch", "signal": "q",  
  "value": 0.0023, "t_start": 0.0, "t_end": 4.670 }
```

3.2.2 Spline Token (per signal)

Emitted for actively moving signals. Uses an adaptive interpolating B-spline: the curve passes exactly through each (ϕ , value) pair. Time is normalized to phase $\phi \in [0,1]$ within the segment, enabling temporal scaling at inference time.

```
{ "type": "spline", "joint": "left_knee", "signal": "q",  
  "t_start": 0.0, "t_end": 9.355, "duration": 9.355,  
  "degree": 3, "cyclic": false,  
  "phase_points": [0.0, 0.18, 0.42, 0.61, 1.0],  
  "values": [0.0, 0.83, 1.52, 0.91, 0.0] }
```

3.2.3 Bundled POSE Token

All active joints at a given phase point ϕ are bundled into one POSE token. Static joints are omitted. This is the primary unit interleaved with Cosmo tokens:

```
{ "type": "pose", "phi": 0.18, "t": 0.84, "duration": 4.67,  
  "cyclic": true,  
  "joints": {  
    "left_knee": {"q": 0.83, "qd": 1.2, "x": 0.04, "y": 0.20, "z": -0.57},  
    "right_knee": {"q": 0.12, "qd": 0.3, "x": 0.04, "y": -0.20, "z": -0.57},  
    ... (active joints only)  
  } }
```

3.3 Adaptive Interpolating Spline Algorithm

The core compression algorithm selects the minimum set of (ϕ, value) pairs such that the interpolating spline reconstructs the full signal within `max_error`. This is a variant of the Ramer-Douglas-Peucker algorithm adapted for splines:

Algorithm: `adaptive_spline(t, v, max_error, degree=3, max_points=300)`

- Seed with endpoints + $(\text{degree}+1)$ evenly spaced interior points
- Fit interpolating B-spline (`scipy.make_interp_spline`) through selected points
- Evaluate reconstruction error on all timesteps
- If $\max(\text{errors}) \leq \text{max_error}$: done
- Else: insert point at $\text{argmax}(\text{errors})$, repeat
- Hard cap at `max_points` to bound token length

Key property: the curve passes **exactly through** every selected point (interpolating), unlike BEAST which uses approximating B-splines where control points attract but do not touch the curve. This makes the tokens directly interpretable as robot states.

3.4 Cyclic Trajectories

Gait signals (knee, hip, ankle) are approximately periodic. When `cyclic=True`, the spline is fitted as a closed curve: the endpoint value matches the start value, and phase wraps at $\phi=1.0 \rightarrow 0.0$. This halves the points needed for repetitive motions and allows the LLM to learn gait periodicity as a structural prior.

Cycle detection: if the signal's autocorrelation has a dominant peak at lag T , and the episode contains ≥ 2 full cycles, the segment is split at cycle boundaries and each cycle is encoded as one cyclic spline token with `"period": T`.

3.5 Configuration

All tokenizer behaviour is controlled by a single `TokenizerConfig` dataclass:

```
TokenizerConfig(
    signals = ["q", "qd", "x", "y", "z"], # modalities to include
    static_thresh = {"q": 0.01, "qd": 0.05, "x/y/z": 0.005},
    max_error = {"q": 0.002, "qd": 0.005, "x/y/z": 0.002},
    spline_degree = 3, # cubic (C2 continuous)
    max_phase_points = 300, # hard cap per signal per segment
    segment_duration = None, # None=whole episode, float=chunk seconds
)
```

Preset configs: `ANGLES_ONLY_CONFIG` (q+qd only, no FK needed), `SPATIAL_ONLY_CONFIG` (x,y,z only), `DEFAULT_CONFIG` (all 5 signals). The spatial config is for models that cannot infer FK from joint angles; the angles-only config is for models with built-in kinematic understanding.

4. Forward Kinematics Pipeline

Pelvis-relative x,y,z positions for each joint are computed offline using **pinocchio** with the Unitree H1 URDF (auto-downloaded via `robot_descriptions`):

- **Input:** q ($T \times 19$ joint angles) from Isaac Sim rollout
- **Joint reordering:** numpy joint order $\rightarrow$ pinocchio order via precomputed remap index
- **FK:** `pin.forwardKinematics` + `pin.updateFramePlacements` per timestep
- **Origin:** pelvis frame position subtracted from all joint positions
- **Output:** $\{ \text{time_key: } \{ \text{joint_name: } \{x, y, z\} \} \}$ for all T timesteps

The pelvis is always the origin (0,0,0). World-space absolute position is not stored in the current rollout format but can be added by capturing `robot.get_world_poses()` in `unitree_episode_logger.py`. For the tokenizer, pelvis-relative positions are sufficient for all pose-understanding tasks.

4.1 Joint Name Mapping (Isaac Sim $\rightarrow$ Pinocchio)

Isaac Sim and pinocchio use different joint orderings. The remap is:

Pinocchio idx	Joint name	Isaac Sim idx
0	left_hip_yaw	0
1	left_hip_roll	3
2	left_hip_pitch	7
3	left_knee	11
4	left_ankle	15
5	right_hip_yaw	1
6	right_hip_roll	4
7	right_hip_pitch	8
8	right_knee	12
9	right_ankle	16
10	torso	2
11	left_shoulder_pitch	5
12	left_shoulder_roll	9
13	left_shoulder_yaw	13
14	left_elbow	17
15	right_shoulder_pitch	6
16	right_shoulder_roll	10
17	right_shoulder_yaw	14
18	right_elbow	18

5. Cosmo Perception Tokens

Perception modalities (video, images, audio) are tokenized separately as **Cosmo tokens** and interleaved with PAB-Spline action tokens. These are *not* action tokens — they represent what the robot perceives or imagines, not what it does.

5.1 Video / Image Tokens

- **Encoder:** Video diffusion VAE (Cosmos, Wan, LTX, or compatible)
- **Input:** RGB frame or image at native camera resolution
- **Output:** Latent vector $z \in \mathbb{R}^d$ (continuous) or VQ-VAE discrete codes
- **Types:** observation frame, goal image, subgoal image
- **Multi-view:** VAE latents from multiple cameras concatenated in T-shape (per LingBot-VA: head cam + left/right wrist cams), encoded separately to avoid cross-view interference

5.2 Audio Tokens

- **Domain:** Frequency domain (mel-spectrogram or FFT coefficients)
- **Periodicity:** Audio is periodic — B-spline compression applies to spectral envelopes
- **Token format:** Cosmo token (VAE latent of spectrogram patch) or raw mel-band values

5.3 Goal and Subgoal Image Conditioning

Following PI's subgoal conditioning approach and andreaskoepf's recommendation, the model is conditioned on:

- **Current observation:** what the robot sees now
- **Subgoal image:** imagined intermediate state (LLM-generated from current frame + text plan)
- **Goal image:** final target state

Subgoal images can be generated by the model itself during planning, conditioned on current video context + actuator history + language instruction. This higher↔lower, planning↔execution interface is the primary design space for VLA generalization.

5.4 Interleaving Schema

Cosmo tokens and POSE tokens are aligned on a common time grid. Video arrives at ~30fps, joints at 200Hz — the spline phase points are chosen to align with video frame boundaries where possible:

```
t=0.000: [COSMO video_t0] [POSE φ=0.00]
t=0.033: [COSMO video_t1] [POSE φ=0.18]
t=0.067: [COSMO video_t2] [POSE φ=0.42]
t=0.100: [COSMO subgoal] [POSE φ=0.61]
t=0.133: [COSMO goal] [POSE φ=1.00]
```

6. Multi-Agent OS Architecture

The execution layer is a parallel multi-agent system where each agent runs asynchronously and communicates via message passing and function calling. This is distinct from the LLM's inference loop.

Agent	Responsibility	Preemption priority
Safety	CBF-verified collision/limit checking Blend stop/avoidance spline in real-time	HIGHEST — preempts all others
Motion	Decode POSE tokens → joint commands Execute spline trajectory at hardware rate	Normal
Vision	Run perception backbone Generate Cosmo tokens Detect subgoal completion	Normal
Exploration	Route new tasks to Motion Update task manifest Request new VLM planning cycle	Lowest

6.1 Fork / Preemption Logic

When the Safety agent detects a constraint violation (obstacle, joint limit, emergency stop), it executes a **Fork**:

- Blend a pre-computed safe spline into the active trajectory at the current ϕ
- No new VLM inference required — operates at sub-millisecond granularity
- CBF (Control Barrier Function) weights are auditable — suitable for aviation-grade certification

This contrasts with end-to-end VLA preemption which requires a full 200–500ms VLM re-inference cycle. The Fork architecture provides formal safety guarantees without sacrificing high-level generality.

6.2 Source Agnosticism

The tokenizer and agent architecture are source-agnostic. The "source" field in each token records the originating system but does not change processing:

```
source: "isaacsim" → Isaac Sim physics rollout
source: "rl_policy" → RL policy trained on real hardware
source: "mocap" → motion capture retargeted to H1
source: "hardware" → real Unitree H1 sensor stream
source: "hf_h1" → HuggingFace Humanoid-Everyday-H1 dataset
```


7. Data Pipeline

7.1 Isaac Sim Collection (current)

- `h1_standalone.py` → runs H1FlatTerrainPolicy, logs q, qd per step
- `unitree_episode_logger.py` → saves .npy rollout + _meta.json
- `compute_joint_positions.py` → FK via pinocchio → pelvis-relative x,y,z
- `pab_spline_tokenizer.py` → adaptive spline tokens → .jsonl

7.2 To add root pose (world-space position)

Modify `unitree_episode_logger.py` to also capture:

```
root_pos = robot.get_world_poses()[0] # (T,3) world x,y,z
root_quat = robot.get_world_poses()[1] # (T,4) quaternion
root_lin_vel = robot.get_linear_velocity() # (T,3)
root_ang_vel = robot.get_angular_velocity()# (T,3)
```

This is optional for pelvis-relative tokenization but required for world-space trajectory analysis.

7.3 HuggingFace Integration

- `build_pairs_manifest.py --mode sample --include-h1-hf --h1-max-rows 500`
- Pairs Isaac Sim rollouts with USC PSI Lab Humanoid-Everyday-H1 HuggingFace dataset
- Output: JSONL manifest for training

7.4 Compression Results (H1 walk, 1871 timesteps, 9.36s)

Config	Tokens	Phase points	Raw values	Ratio
q only	19	4,822	35,549	7.4×
q + qd	38	10,617	71,098	6.7×
q + qd + xyz	95	~24,000	177,745	~7.4×
with static joints	varies	fewer	same	8–15×

8. Implementation Status and Next Steps

Component	Status	File
Isaac Sim H1 rollout logger	✓ Complete	unitree_episode_logger.py
FK pipeline (pinocchio)	✓ Complete	compute_joint_positions.py
Adaptive spline tokenizer	✓ Complete	pab_spline_tokenizer.py
Static token detection	✓ Complete	pab_spline_tokenizer.py
Reconstruction / decode	✓ Complete	pab_spline_tokenizer.py
Compression stats	✓ Complete	pab_spline_tokenizer.py
Bundled POSE token format	■ TODO	pab_spline_tokenizer.py v2
Cyclic gait detection	■ TODO	pab_spline_tokenizer.py v2
Root pose logging	■ TODO	unitree_episode_logger.py
Cosmo video tokenizer	■ TODO	cosmo_tokenizer.py
Audio frequency tokenizer	■ TODO	cosmo_tokenizer.py
Cosmo ↔ POSE interleaving	■ TODO	sequence_builder.py
HuggingFace dataset upload	■ TODO	build_pairs_manifest.py
LLM training integration	■ TODO	—
Multi-agent OS (Safety/Motion)	■ TODO	—

8.1 Immediate Next Steps (in order)

- **v2 tokenizer:** implement bundled POSE token output + cyclic spline detection
- **Cosmo tokenizer:** wrap Cosmos/Wan VAE for video frame → latent token
- **sequence_builder.py:** interleave Cosmo + POSE tokens on common time grid
- **Re-run sim:** add root_pos/root_quat to episode logger, collect longer episodes
- **HuggingFace:** upload tokenized episodes alongside FineVideo-VLA-Agent dataset
- **LLM training:** test with 25B tokens per Huu's suggestion (~45 min), load to HuggingFace

9. Key Design Decisions and Rationale

9.1 Interpolating vs Approximating Splines

Unlike BEAST (approximating B-splines with control points that attract but do not touch the curve), PAB-Spline uses **interpolating splines** where the curve passes exactly through each selected point. This means every phase point is a directly interpretable robot state, making tokens meaningful to both the model and human engineers.

9.2 Why Phase $\phi \in [0,1]$ Instead of Absolute Time

Phase normalization decouples trajectory shape from execution speed. The same POSE token sequence can be replayed at 0.5× or 2× speed by adjusting the `duration` field at decode time. This enables temporal scaling as a first-class feature — critical for adapting pre-computed motions to different task speeds without re-inference.

9.3 Why Pelvis-Relative Positions

Pelvis-relative x,y,z positions are translation-invariant and rotation-invariant with respect to the robot's location in the world. This means the same pose token is valid regardless of where the robot is standing, enabling better generalization. Models that struggle to infer spatial layout from joint angles alone benefit from explicit xyz — hence the configurable `signals` field.

9.4 Static Token Compression

For many tasks, large subsets of joints are stationary for extended periods (arms during walking, legs during manipulation). Static tokens eliminate these from the active token stream entirely, replacing $N \times T$ values with a single token. Compression ratio increases significantly with task specificity: walk-only episodes compress 7–10×; manipulation episodes where arms are static may compress 15×+.

9.5 Source Agnosticism

The tokenizer does not care whether q/qd comes from Isaac Sim, an RL policy, real hardware, or MoCap retargeting. The `"source"` metadata field records provenance but does not affect tokenization. This is essential for building a dataset that mixes simulation and real-world data without architectural changes.

10. File and Module Reference

File	Purpose
h1_standalone.py	Isaac Sim entry point. Spawns H1, runs policy, calls logger.
unitree_episode_logger.py	Records q, qd per step. Add root_pos/root_quat for world-space.
logs.py	Legacy logger (has base_position). Not currently used in h1_standalone.
data_schema.py	EpisodeRecord dataclass + JSONL save/load + validation.
check_data.py	Quick inspection script for .npy rollout files.
build_pairs_manifest.py	Pairs local Isaac Sim rollouts with HuggingFace H1 dataset.
compute_joint_positions.py	FK pipeline: q → pelvis-relative x,y,z via pinocchio + H1 URDF.
pab_spline_tokenizer.py	Main tokenizer. adaptive_spline, tokenize_episode, reconstruct.
cosmo_tokenizer.py	TODO: video/audio → Cosmo latent tokens.
sequence_builder.py	TODO: interleave Cosmo + POSE tokens on common time grid.